

Lab 5

Microprocessor Architectures [ELEC-H-473]

Superscalar Execution & Cache Memory

v1.1.1

1 Introduction

This laboratory is divided in two parts, the first is about superscalar CPU architecture & execution and the second is about memory cache. The goals are to:

- **Part 1:** Understand how instructions are executed in a superscalar CPU architecture
- **Part 1:** Write and test some programs in assembly code against this architecture
- **Part 1:** Explore how the different micro-architectural elements of a CPU influence workload execution
- **Part 2:** How the cache memory architecture influence the data hit/miss rate & memory access time

The tools used in this labs are open source.

- **Part 1:** Download the folder named **freess**, available on the UV. The paper related to this tool can be found here . Be aware that the online version of the tool does not cover all the parameters and can be prone to bugs.
- **Part 2:** The second one can be found on GitHub, but we'd advise you to download the version available on the UV as we've updated the code to python 3.X.

2 Part 1: Superscalar CPU Execution

To get a deeper understanding of a superscalar CPU inner workings, you will use a simulator designed by Roberto Giordi from the University of Sierra, Italy. The full processor description, as well as execution examples are explained in his paper. Your very first task is to read it and get yourself familiar with the different architectural parts, but we will quickly and coarsly go over them here. We particularly recommend to read the section 3, as it describes the architecture and the execution pipeline.

2.1 Processor Description

This processor is, like the previous labs, a pipelined one. The pipeline stages differs from the previous simulators as it is superscalar, and out-of-order. Dynamic pipeline scheduling is used to choose which instruction is executed in a given clock cycle while trying to avoid hazards and stalls by re-ordering them. Conceptually you can think of a dynamically scheduled pipeline as analysing the dataflow structure of program, and re-ordering the instruction order to minimize hazards while preserving the data flow of the program. The architecture is show in Figure ??

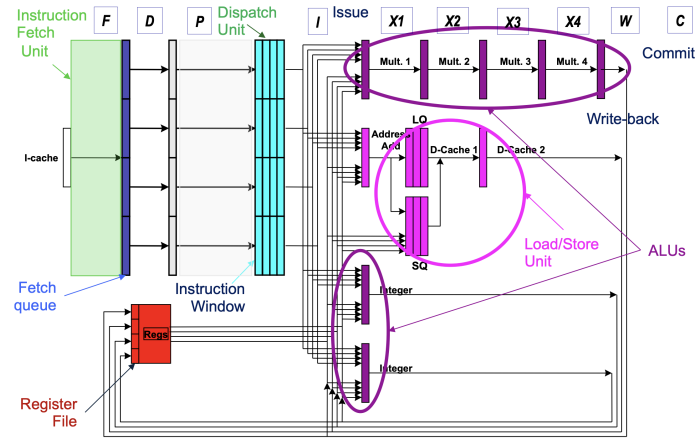


Figure 1: The superscalar architecture of FREESS

The pipeline has 7 stages and instructions are executed as follow:

1. **F (Fetch)**: Instructions are read from memory, multiples at once.
2. **D (Decode)**: Multiple instructions are decoded during the clock cycle
3. **P (Dispatch)**: Depending on the operation, an instruction is affected to the reservation station/instruction window of the proper execution unit
4. **I (Issue)**: After dispatching instructions are queued to be issued to the appropriate execution unit
5. **X (Execute)**: ALU/MUL unit process the instruction
6. **W (Write-back)**: Result is written into the reorder buffer (ROB) and a *c* flag is set to 1.
7. **C (Commit)**: Decide when the instruction is safe to be released to a programmer-visible register or memory

The instruction execution can be monitored in the terminal through the following interface, shown in Figure ??, more detail is available in the paper.

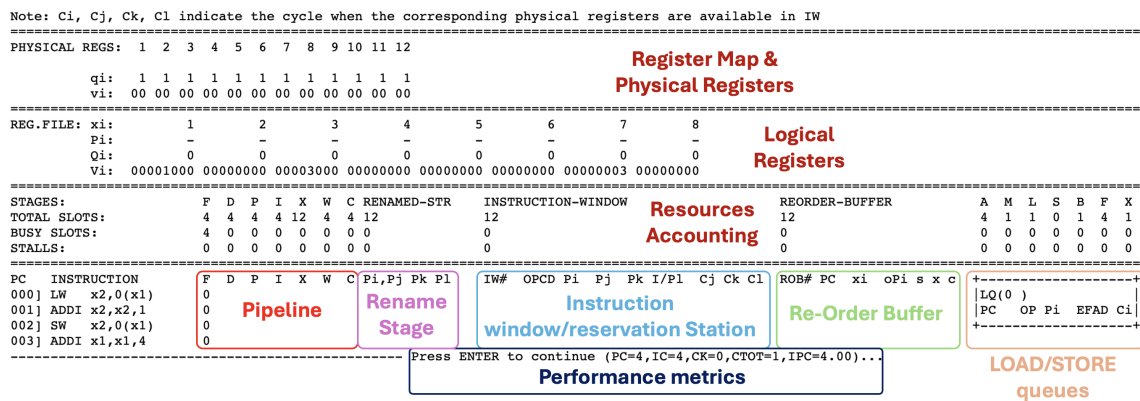


Figure 2: Pipeline view in terminal

This CPU has a reduced number of instructions (8), and foregoes the use of an assembler. Table 1 summarize the opcodes.

Fortunately, we provide you with the required code. One being the fibonacci sequence provided in lab 1, and another being a multiplication algorithm from the same lab.

InstructionOpcode	
LW	1
SW	2
BEQ	3
BNE	4
ADD	5
ADDI	6
MUL	7

Table 1: Instructions opcode

FREESS Parameters	Unit	Meaning
-fw	<insts/cycle>	Fetch Width: # of instr fetched in 1 CC
-dw	<insts/cycle>	Decode Width: # of instr decoded in 1 CC
-iw	<insts/cycle>	Issue Width: # of instr issued in 1 CC
-cw	<insts/cycle>	Commit Width: # of instr committed in 1 CC
-wins	<insts>	# of slots in Instruction Window (IW)
-robs	<inst>	# of slots in Re-Order Buffer (ROB)
-rreg	<num>	# of logical registers
-preg	<num>	# of physical registers
-lqs	<insts>	# of slots in load queue
-lfu	<num>	# of load functional units
-llat	<cycle>	# of CC to perform load operation
-sqs	<insts>	# of slot in store queue
-sfu	<num>	# of store functional units
-slat	<cycles>	# of CC to perform a store operation
-afu	<num>	# of integer ALU units
-alat	<cycles>	# of CC to perform an ALU operation
-mfu	<num>	# of integer MUL units
-mlat	<cycles>	# of CC to perform a MUL operation

Table 2: Modifyable Superscalar parameters

One last thing we will cover here is the μ -architectural parameters. FREESS allow you to modify its μ -architecture, altering performances.

You can always play around with the different parameters and see how they affect the program execution speed. Try to increase them a lot (Apple M4 has a fetch/decode width of 10!) to understand the limitations of the different parameters and how they affect the workload execution and interact with the other stages.

Run the simulator: To run the simulator, you will need to

1. Compile the test programs then check if it works correctly

```
make
make test
```

2. To run your own test with specific μ -Arch parameters
 - (a) Open **TestMakefile.mk** and change the parameters you want
 - (b) Create a program file with your code (no extension needed on the file)
 - (c) Run with

```
make -f TestMakefile.mk PROGRAM=yourProgramFileName
```

Question 1: Use the provided, translated, example of the first lab, detail the state of the different stages & how the PC evolves during the first X clock cycles. Make sure to understand what happens at every stage.

Question 2: What is the maximum theoretical IPC for a superscalar processor ? What will worsen it and how could you improve the IPC ?

Question 3: Change the CPU fetch/decode width (separately, then simulatenously), how do the hazards changes ? Why ? Does it create a bottleneck ?

Question 4: Take the second code provided, run it with the default configuration and write down the key metrics. Play around with the FREESS parameters (fetch, decode, ROB, etc) to optimize the architecture for that workload. How does the performance metrics changes ?

3 Part 2: Cache Memory

Before being able to run the code, you must install two python packages: `pyyaml` and `terminaltables`. The cleanest way to do so is to create a virtual python environment, which will contain "locally" the required packages. Enter the following command in your terminal, while being inside the `CacheSimulator` folder:

```
python3 -m venv .h473_venv
source .h473_venv/bin/activate
pip install pyyaml
pip install terminaltables
```

We expect you to already be a bit familiar with the basic concepts of cache memory such as the different field in a cache, what things such as hit time, miss penalty, miss rates, are etc...

3.1 Cache Performance

The goal of this part is to get yourself more familiar with the inner workings of cache memories and the impact their architectures may have on performance. Figure ?? shows the architecture of a direct-mapped cache:

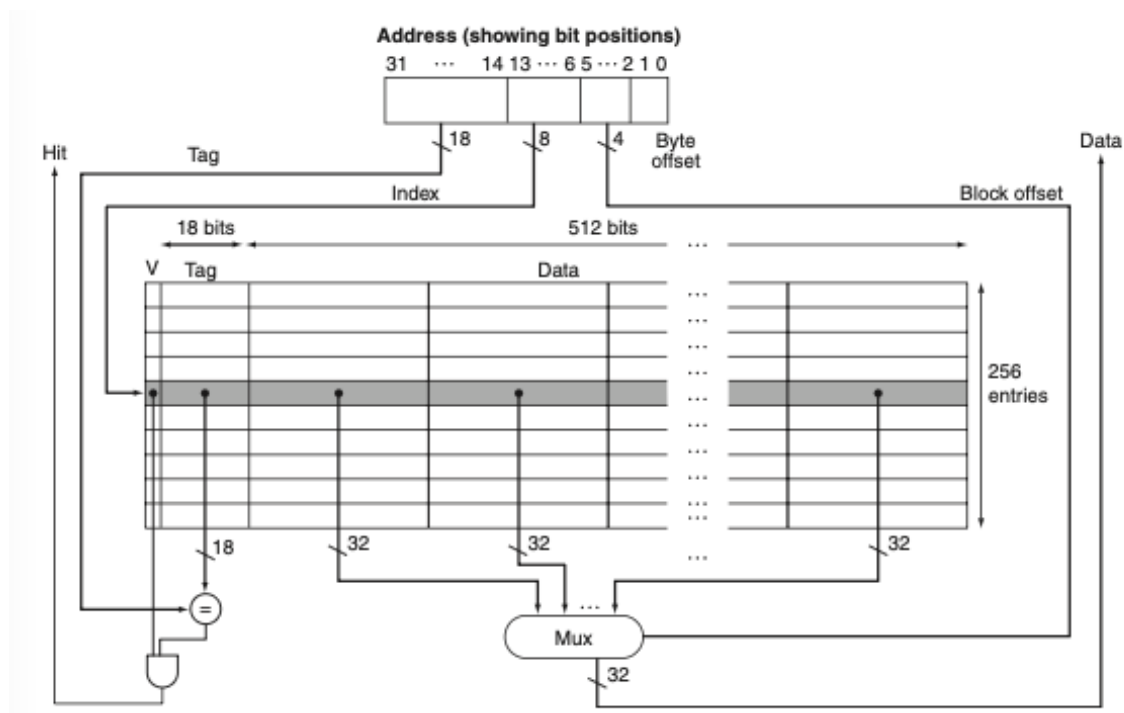


Figure 3: Direct mapped cache memory with 16 words per block. The cache has 256 entries. On a 32 address, 18 bits are used for the tag, 8 bits for the index, 4 bits for the block offset and 2 for the byte offset.

For a given trace file, assuming a 32 bit (words) architecture and data transfer with main memory:

Question 1: What is the role of the different field in a cache address ?

Question 2: Express the average memory access time for a single level of cache and, after that, a multi-level cache (assume 2) ?

Question 3: Using the simulator, for a direct mapped cache configuration, sweep the block size & plot the miss rate. Why do you see such a behavior regarding miss rate?

Question 4: Using the simulator, for a fixed block size, sweep the associativity. What behavior do you see, what "rule of thumb" can you derive ? How does the different cache address fields evolve with associativity ?

Question 5: What does it means to have a fully associative cache ? What is the impact on hardware ?

Question 6: What are the three big types (3 Big "C's"), of cache miss ? How could you optimise your cache for them ?